

Reviewer Pack — Noncommutative Geometry of Boolean Functions Bounds Resoluti...

Entry f281162f1654 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 18:34:13 UTC

Noncommutative Geometry of Boolean Functions Bounds Resolution Proof Length

Entry ID: f281162f1654 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 18:34:13 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative geometry **Field B** (complexity object): Resolution proof length

Statement:

For every CNF φ with m clauses and n variables, the norm of the noncommutative space associated with φ , denoted as $N(\varphi)$, is upper bounded by a polynomial in $\log(m)$ and $\log(n)$, i.e., $N(\varphi) = \Theta(\log^2(m + n))$.

Rationale (proposer's reasoning):

Noncommutative geometry has been used to study quantum systems, which may provide new insights into the structure of Boolean functions. This conjecture proposes that the resolution proof length of a CNF can be bounded by the noncommutative space associated with it, suggesting a potential link between noncommutative geometry and computational complexity.

Taxonomy category: NONCOMMUTATIVE_GEO (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0b7c64ccb61a1fb2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the average norm of noncommutative spaces for all CNFs exceeds the lower bound by at least 95% confidence with $p\text{-value} < 0.05$, and no CNF has a norm exceeding its upper bound by more than 10%. The conjecture is falsified if any CNF violates the upper bound by more than 10% or if the correlation coefficient between $N(\varphi)$ and resolution proof length is negative.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:noncommutative geometry AND resolution proof length - polynomial bound noncommutative geometry resolution proof - norm CNF ϕ upper bounded $\log^2(m+n)$

Top relevant hits considered: - [http://arxiv.org/abs/astro-ph/0407622v3] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly's Eye - [http://arxiv.org/abs/2603.03026v1] Any Resolution Any Geometry: From Multi-View To Multi-Patch - [http://arxiv.org/abs/math/0205005v2] The rings of noncommutative projective geometry - [http://arxiv.org/abs/math/0610043v5] Lecture Notes on Noncommutative Algebraic Geometry and Noncommutative Tori - [http://arxiv.org/abs/1304.4062v2] Superconformal nets and noncommutative geometry - [http://arxiv.org/abs/1209.3595v2] Noncommutative complex differential geometry - [http://arxiv.org/abs/1803.07376v1] Sub-exponential Upper Bound for #XSAT of some CNF Classes - [http://arxiv.org/abs/1004.0536v7] Upper bounds on the solutions to $n = p + m^2$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(m, n):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def norm_of_noncommutative_space(cnf):
        # Placeholder implementation of the noncommutative space norm
        m = len(cnf)
        n = max(abs(lit) for clause in cnf for lit in clause)
        return math.log(m + n, 2)**2

    def resolution_proof_length(cnf):
```

```

    # Placeholder implementation of the resolution proof length
    m = len(cnf)
    n = max(abs(lit) for clause in cnf for lit in clause)
    return m * n

results = []
for _ in range(30): # Aim for at least 30 instances per seed
    m = random.randint(5, 40)
    n = random.randint(5, 40)
    cnf = generate_cnf(m, n)
    norm = norm_of_noncommutative_space(cnf)
    proof_length = resolution_proof_length(cnf)
    results.append((norm, proof_length))

if not results:
    return {
        "metric_name": "N( $\phi$ )",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

norm_values = [norm for norm, _ in results]
proof_lengths = [proof_length for _, proof_length in results]

mean_norm = sum(norm_values) / len(norm_values)
std_norm = math.sqrt(sum((x - mean_norm)**2 for x in norm_values) / len(norm_values))
correlation_coefficient = sum((norm_values[i] - mean_norm) * (proof_lengths[i] - mean(proof_lengths))) / (len(norm_values) * std_norm)

conjecture_holds = all(mean_norm >= 0.95 * norm and norm <= 1.1 * norm for norm, _ in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "N( $\phi$ )",
    "metric_value": mean_norm,
    "instances_tested": len(results),
    "n_max": 40,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

```

```

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=budget_exceeded n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e9eaad72.py", line 84, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e9eaad72.py", line 64, in run_trial
    correlation_coefficient = sum((norm_values[i] - mean_norm) * (proof_lengths[i] -
mean(proof_lengths)) for i in range(len(results))) / (len(results) * std_norm * math.sqrt(sum((x -
mean(proof_lengths))**2 for x in proof_lengths)))

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e9eaad72.py", line 64, in <genexpr>
    correlation_coefficient = sum((norm_values[i] - mean_norm) * (proof_lengths[i] -
mean(proof_lengths)) for i in range(len(results))) / (len(results) * std_norm * math.sqrt(sum((x -
mean(proof_lengths))**2 for x in proof_lengths)))

```

NameError: name 'mean' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21722
2	preregistration	ollama_remote	glm4:latest	0	0	11275
3	novelty	ollama_remote	glm4:latest	0	0	12417
4	novelty	ollama_remote	glm4:latest	0	0	20836
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14723
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19788
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23941
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	103603
9	judge	ollama_remote	glm4:latest	0	0	10695

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 238999 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/f281162f1654.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/f281162f1654.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/f281162f1654.tar.gz* (if generated)